Design the Uber API

SystemsExpert design walkthrough

Clarifying questions

- Rider and driver APIs both?
- Payments in scope?
- Real-time location updates required?

Functional requirements

- Rider: request ride, see ETA/driver location, cancel, rate
- Driver: go online/offline, accept ride, update location, complete trip

Non-functional requirements

- Real-time updates (~seconds)
- Idempotent state transitions

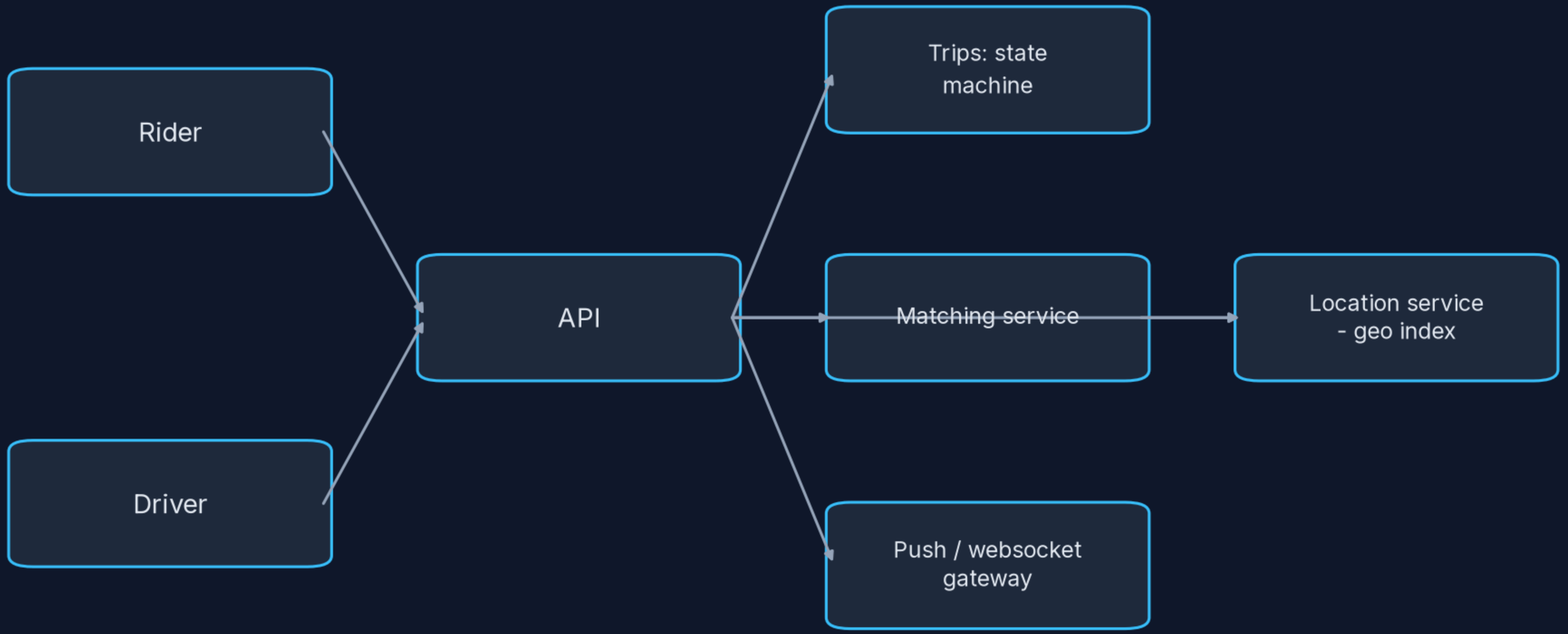
Capacity estimates

- Drivers send location every ~4 s; ~1M active drivers → ~250k writes/s to the location service

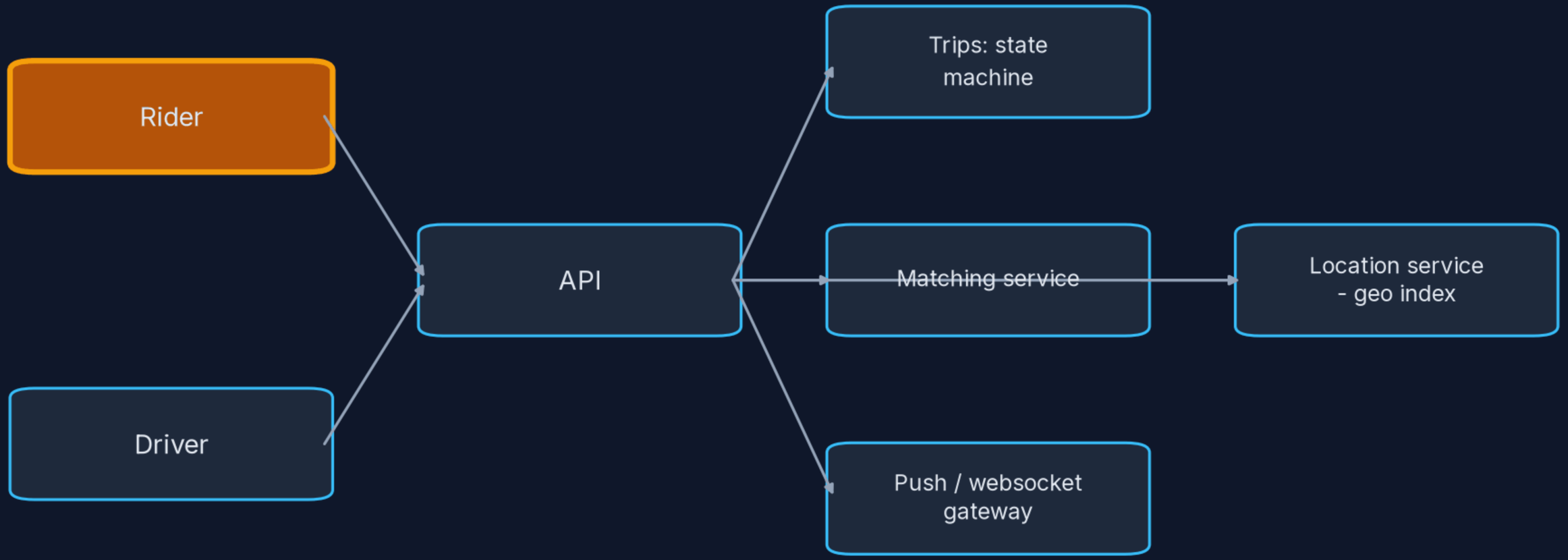
Capacity: worked out

- 1M active drivers, 1 location update / 4 s
- 1M ÷ 4 = 250k writes/s
- Keep only latest position in memory (~100 B)
- 1M × 100 B = 100 MB → shard by geo cell

High-level architecture

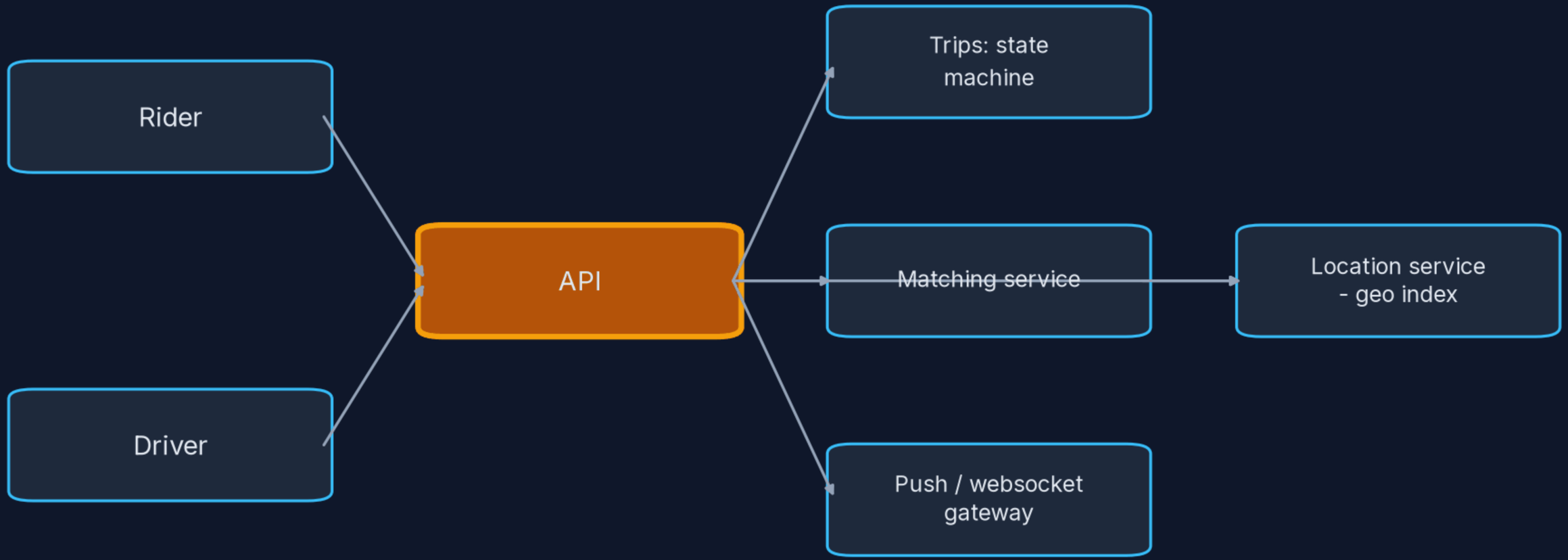


Request flow: Request a ride



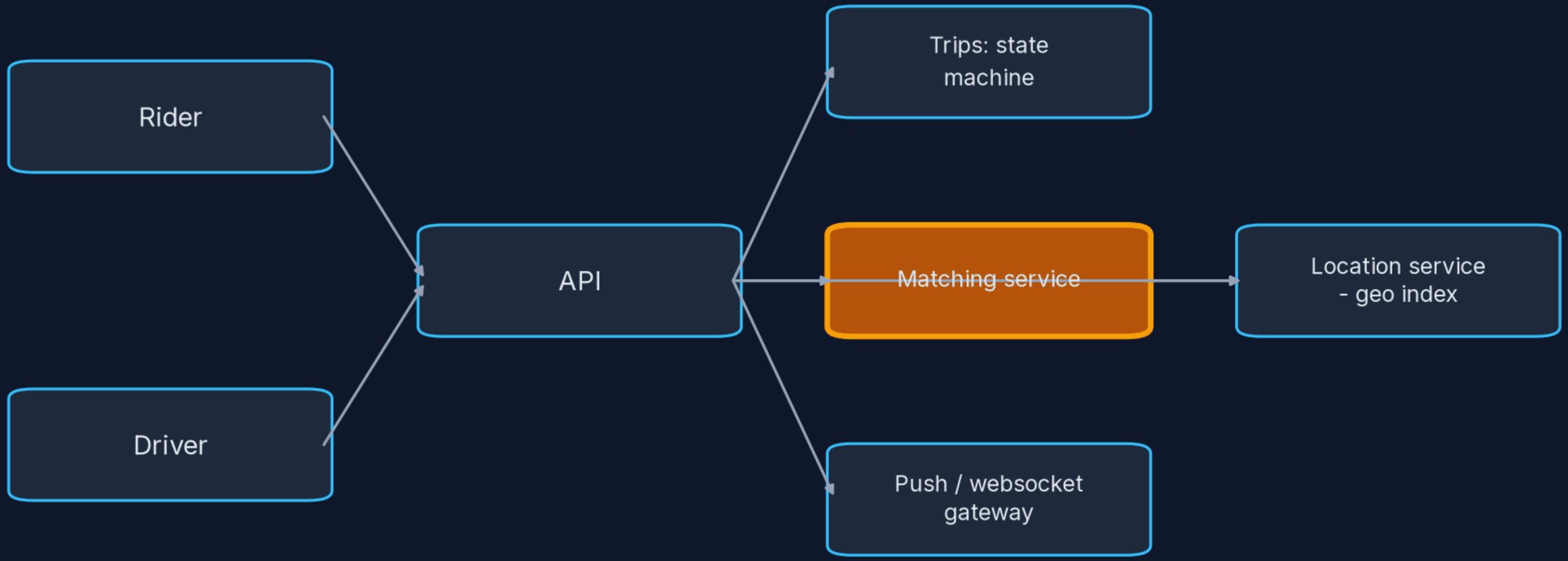
1. Rider requests a ride

Request flow: Request a ride



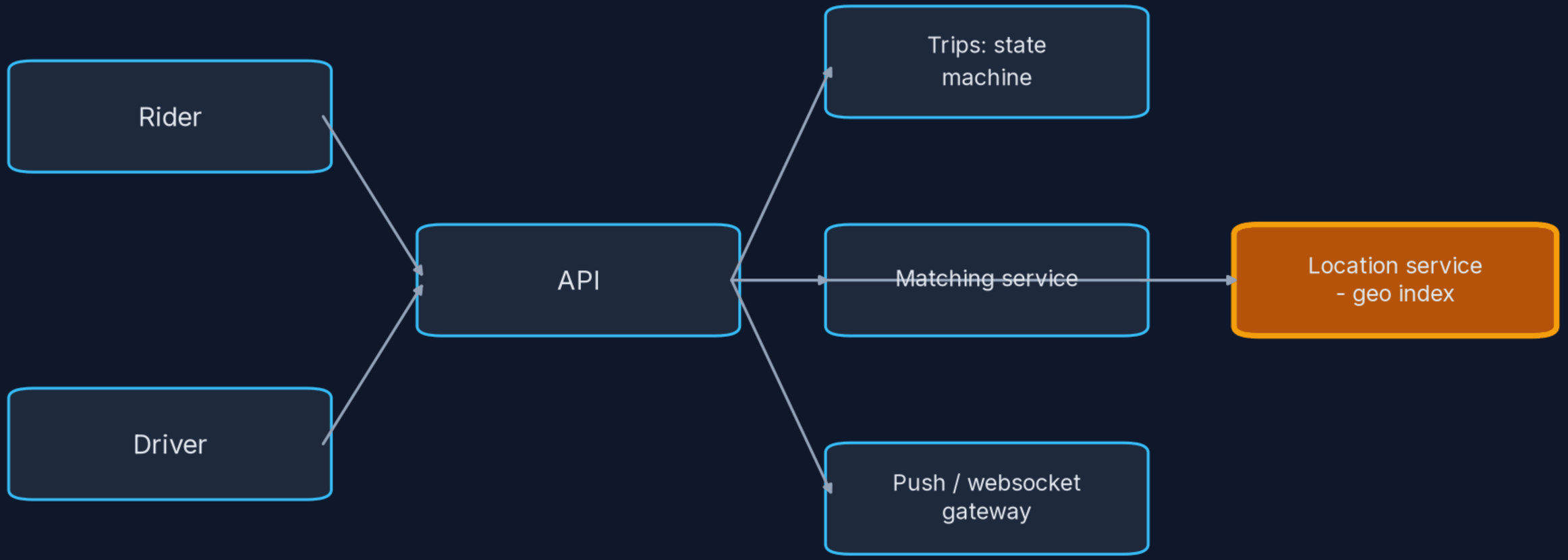
2. API creates the trip

Request flow: Request a ride



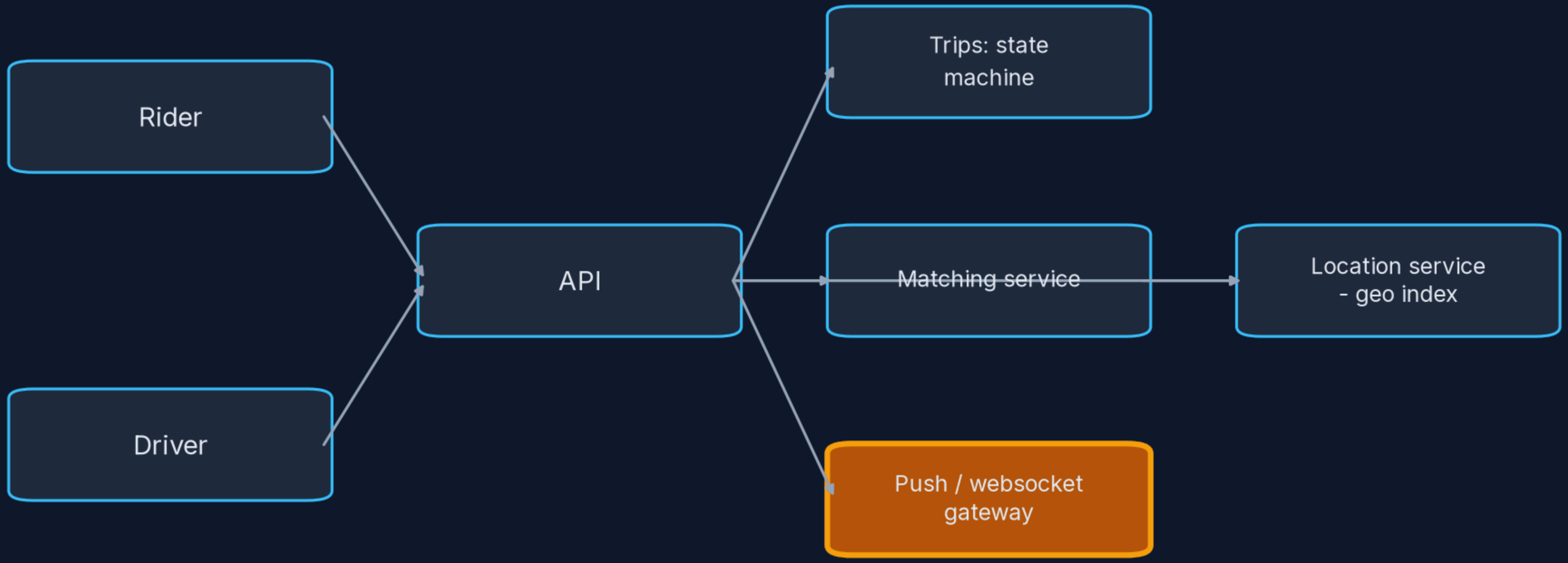
3. Find nearby drivers

Request flow: Request a ride



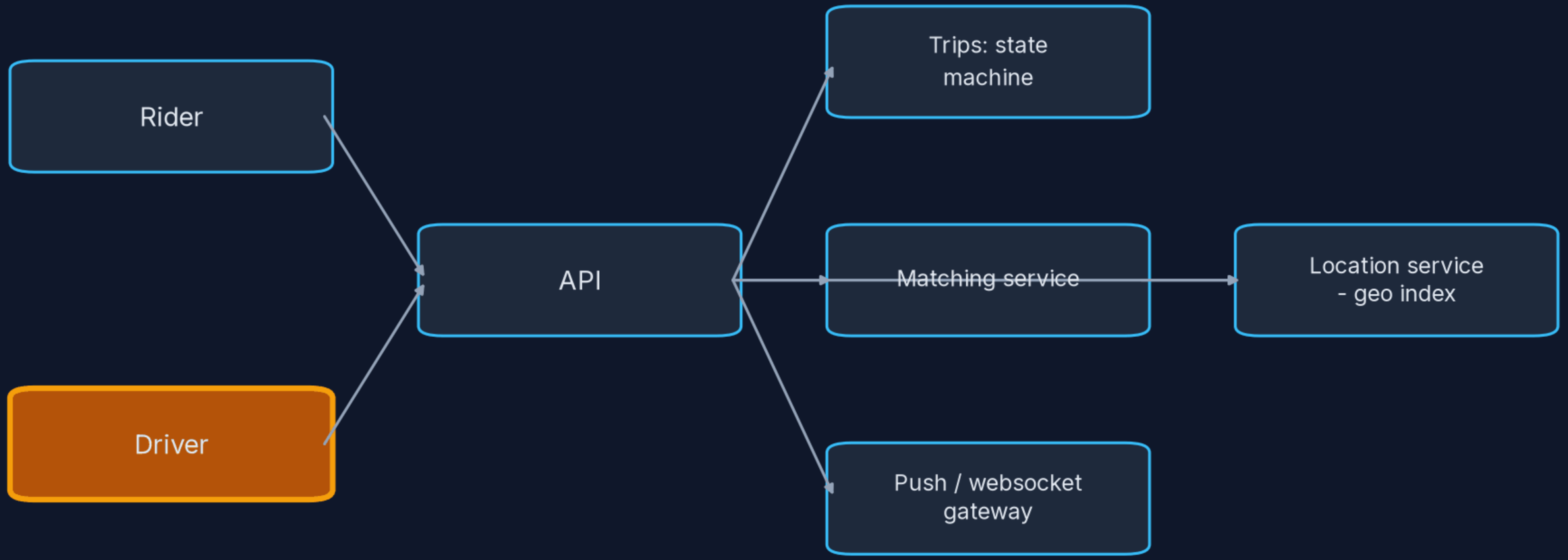
4. Geo index lookup

Request flow: Request a ride



5. Offer sent to a driver

Request flow: Request a ride



6. Driver accepts

Data model

- Trip{tripId, riderId, driverId?, status: REQUESTED|ACCEPTED|ARRIVED|IN_PROGRESS|COMPLETED|CANCELLE pickup, dropoff, fare}
- DriverStatus{driverId, online, location, tripId?}

API

```
POST   /trips                          {pickup, dropoff, idempotencyKey} -> Trip
GET    /trips/{tripId}                 -> Trip incl. driver location, ETA
POST   /trips/{tripId}/cancel
POST   /trips/{tripId}/rating          {stars}
PUT    /drivers/me/status              {online: true|false}
PUT    /drivers/me/location            {lat, lng}
POST   /trips/{tripId}/accept
POST   /trips/{tripId}/arrive | /start | /complete
Stream: GET /stream (websocket) -> trip updates, offers to drivers
```

State machine

- Every transition is validated server-side (e.g. accept only from REQUESTED) with optimistic concurrency, so double taps or two drivers accepting resolve deterministically.

Location

- Updates go to an in-memory geo index (geohash/S2 cells) sharded by region; matching queries nearby cells.

Push vs poll

- Websocket for live location; polling fallback.

Key decisions

- In-memory geo index → write rate too high for a DB
- Trip as a state machine → no double assignment
- WebSockets for drivers → low-latency offers
- Shard by city / geo cell → locality of matching

Trade-offs & alternatives

- Separate endpoints per transition are explicit and auditable vs a generic PATCH status
- In-memory geo index trades durability for speed (locations are ephemeral anyway)

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: Rider, API, Driver, Trips, Location service - geo index
- Know your trade-offs and failure modes